

ПРАКТИЧЕСКАЯ РАБОТА № 8.1

Тема: Технологии проверки целостности данных и резервирование: надежность данных в ОС Linux.

Программное обеспечение:

1. ISO образ ОС Debian

<https://cloud.debian.org/images/archive/12.7.0/amd64/iso-cd/> или ISO образ ОС Kali Linux <https://www.kali.org/get-kali/#kali-platforms>

Вопросы для обсуждения:

1. Что такое целостность данных в контексте операционных систем?
2. Какие ОС бывают в зависимости от степени участия человека (пользователя)? В чем отличие данных ОС?
3. Что относится к основным элементам ОС семейства Unix?
4. Что такое kernel?
5. Что входит в классическую файловую систему ОС семейства Unix?
6. Назовите основные конфигурационные файлы ОС семейства Linux?
7. Что такое «хэширование» и какие существуют алгоритмы хэширования?
8. Что такое Bacula?
9. Что представляет собой утилита Amanda?
10. Что представляет собой утилита dump?

Цель: Приобретение навыков работы с технологией проверки целостности (хеширования) и резервирования (синхронизации) данных в ОС Linux.

Основные теоретические сведения

В настоящее время неотъемлемой частью любого ПК является такое программное обеспечение как операционная система (ОС), которая

представляет собой совокупность системных программ, предназначенных для обеспечения определенного уровня эффективности системы обработки информации за счет автоматизированного управления ее работой и предоставляемого пользователю определенного набора услуг [ГОСТ 15971-90].

В зависимости от степени участия человека (пользователя, администратора) в такой системе ОС бывают:

- процедурная (участие человека в большей степени заключается в выполнении только тех действий, которые изначально заложены в ОС, т.е. точному следованию «инструкции по применению»). Основными представителями данного типа являются ОС семейства Windows;

- проективная (участие человека заключается в использовании собственного инструментария ОС для эффективного решения повседневных или повторяющихся задач пользователя). Основными представителями данного типа являются дистрибутивы ОС Unix.

Основные отличия данных ОС представлены в таблице 1.

Таблица 1 – Сравнение свойств проективных и процедурных ОС

Свойства	Тип операционной системы	
	Проективная	Процедурная
<i>информационная открытость</i>		
доступность информации о механизмах работы ОС;	да	нет
необходимость освоения с документацией для использования ОС;	да	нет
открытые исходные тексты программ;	да	нет
<i>минимизация затрат</i>		
возможность решения мыслительных и логических задач;	стандартными ср-вами ОС	дополн. ПО
вероятность механической ошибки;	нет	да

степень квалификации пользователя;	ориентация на профессионала	ориентация на новичка
<i>умопостижимость контекста</i>		
учет ограниченных возможностей человеческой памяти и восприятия;	да	нет
любой проект должен быть таким, чтобы пользователь мог прочесть его и (после изучения документации) понять целиком (human readable);	да	нет
любой проект возможно не только просчитать, но и дополнить пользовательскими процедурами, функциями;	да	нет
<i>персональная ответственность</i>		
любая элементарная команда работы с системой предполагает, что человек сначала подумал и принял решение	да	нет
ответственность обусловлена знаниями.	да	нет

В рамках данной практической работы будет рассмотрена структура ОС семейства Unix, которая представлена на рисунке 1.

Основными элементами ОС семейства Unix являются:

- ядро;
- модули ядра;
- демоны;
- процессы;
- интерфейс;
- утилиты.

Рассмотрим каждый элемент вкратце.

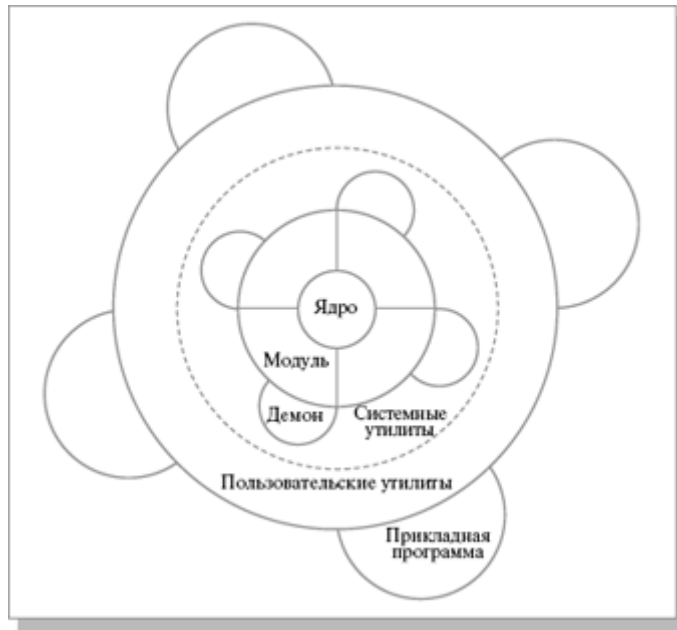


Рисунок 1 – Структура ОС семейства Unix

Ядром системы (kernel) является менеджер ресурсов и планировщик задач, которые постоянно работают в режиме супервизора. Постоянно находясь в памяти, ядро обслуживает непрерывный поток запросов на использование универсальных ресурсов системы (памяти и времени) и предоставляет программам пользователя унифицированный интерфейс к ресурсам компьютера (так называемые системные вызовы, system calls), а также содержит всю непростую логику распределения ресурсов по задачам, которые в Unix называются процессами.

Ядро системы ОС Unix комплектуется базовым набором драйверов самых популярных устройств для поддержки подавляющего количества компьютеров. В случаях отсутствия в комплекте необходимых драйверов существует возможность самостоятельного пересборки (компиляции) ядра системы. Процесс компиляции ядра требует широких знаний об архитектуре используемой версии Unix, архитектуре ЭВМ и особенностях внешних устройств, поэтому сборку ядра необходимо осуществлять только опытным пользователям.

Если базового ядра все-таки недостаточно, в современных системах

многие драйверы можно загружать динамически из *модулей ядра*. Ядро, уже работающее в памяти, можно дополнить, загрузив такой модуль из файла специального формата, после чего перекомпоновать ядро на ходу (специальным компоновщиком). В эти модули можно вынести необязательные функциональности системы (например, фильтрацию сетевых пакетов), после чего базовое ядро станет еще меньше, однако процесс загрузки усложнится, так как некоторые из подгружаемых модулей понадобятся ядру уже при старте системы, когда доступа к файлам может и не быть. Типичный пример: для работы с диском ядру нужен драйвер дискового массива (RAID-контроллера), который вместе с программой загрузки и компоновки модулей на этом массиве и находится. Разные системы выходят из этой ситуации по-разному.

Модули ядра работают в режиме ядра, поэтому обращаться с ними следует крайне осторожно: ошибка в таком модуле столь же фатальна, как и ошибка ядра, и в лучшем случае вызовет крах системы (в худшем случае система ее заметит не сразу).

С ядром взаимодействуют функциональные подсистемы (службы), т.е. наборы программных средств, выполняющих определенную функцию (например, система печати, система передачи почты и т.д.). Управляющий центр функциональной подсистемы является *демон* (daemon, в переводах с греческого называемый "даймон"). Демон запускается при старте Unix для обслуживания запросов к функциональной подсистеме и обменивается данными с ядром системы, а также держит очередь пользовательских запросов, работает с сетью и т.д.

Обращаться к системным вызовам могут, не только демоны, но и любые программы. В Unix входит немало программ, при помощи которых можно решать разнообразные инструментальные (т.е. связанные с работой самой системы) задачи. Это так называемые *системные утилиты*. Они используются в первую очередь самой системой и системным администратором для управления системой. Однако и пользователь, не

обладающий правами администратора, вполне может задействовать системные утилиты, если они помогают ему в работе, а системе не мешают (например, создавать файловую систему на дискете, просматривать состояние системы или демонов и т. п.).

Кроме системных утилит существуют еще пользовательские которые занимаются преобразованием текста и/или помогают при разработке решений: компиляторы, отладчики, редакторы диаграмм, трассировщики и т.д. Почти всеми пользовательскими утилитами пользуется система, потому что при проективном подходе вообще невозможно провести четкую границу между системным и пользовательским наполнением. К обеим категориям, например, относятся утилиты для работы с файлами и файловой системой или интерпретатор командной строки (shell). На shell написаны все системные сценарии, поскольку он представляет собой еще и удобный высокоуровневый язык программирования.

В роли задач в Unix выступают *процессы* – программы, запущенные пользователем, которые находятся в памяти и потребляют ресурсы на: выполнение, выделение памяти, обмена данными с системой, внешними устройствами и другими процессами. При запуске процесс получает уникальный идентификатор процесса (Process Identifier, PID), по которому он становится доступен другим процессам и планировщику.

Между собой процессы могут обмениваться данными не только стандартными пользовательскими средствами (посредством файлов, каналов или сокетов), но и с помощью более быстрых системных, именуемых средствами межпроцессного взаимодействия (Interprocess Communication, IPC). Процессы могут заказать у системы общую память (тогда часть адресного пространства каждого будет ссылаться на один и тот же кусок реальной памяти), для индикации занятости ресурса использовать семафор (система гарантирует, что запрошенный ресурс действительно будет свободен, пока процесс не откроет семафор) и посылать друг другу сигналы и сообщения.

Основным инструментом диалога с пользователем в Unix выбрана командная строка, который осуществляет запуск вводимой команды с клавиатуры. Команды могут быть совсем короткими (одно нажатие), могут содержать имя запускаемой утилиты и несколько коротких параметров, а могут быть даже небольшими программами (символов в сто). Команды большего размера неудобно вводить и исправлять прямо в командной строке, их стоит складывать в файл, называемый сценарием (script). Такой сценарий тоже считается программой, его можно вызывать по имени, передавать параметры и т. д. Все эти команды распознает и выполняет интерпретатор командной строки (shell, "оболочка"), в который встроены специальные возможности, помогающие очень быстро набирать командную строку и оперативно объединять, и использовать результаты выполнения других программ.

Для обозначения точки входа в систему - места, откуда приходят команды, и куда следует выдавать результат их работы, используется понятие терминала - устройства способного принимать и передавать текстовую информацию. Раньше было великое множество таких устройств, они подключались к последовательному порту компьютера и состояли из клавиатуры и устройства вывода текста (либо принтера, либо так называемого алфавитно-цифрового монитора, т.е. монитора, который умеет выводить буквы, цифры и некоторые символы). Сейчас с этой задачей справляются специальные драйверы.

Частью *интерфейса* Unix по праву можно считать файловую систему, основной функцией которой является предоставление организованного пространства для хранения файлов. Кроме того, файловые системы обеспечивают определение пространства имен, которое определяет такие параметры как название файлов, ограничение на длину имени, используемые символы, логическую структуру диска и т.д.

Классическую файловую систему семейства Unix можно представить в виде простой иерархии, в которой все каталоги находятся в корневом

каталоге (/). Полные имена (пути) всех остальных каталогов получаются из «/», к которому дописывается справа имена последовательно вложенных друг в друга каталогов. В корневом каталоге обычно находятся только подкаталоги со стандартными именами. Более того, не только имена, но и тип данных, которые могут попасть в тот или иной каталог регламентированы стандартом Filesystem Hierarchy Standart («Стандартная структура файловых систем»). Стандартное размещение файлов позволяет человеку и программе предсказать, где находится тот или иной компонент системы.

В настоящее время большинство дистрибутивов Unix имеют следующую структуру:

/	–	корневой каталог
/bin	–	(binaries) бинарные файлы пользователя
/boot	–	статичные файлы системного загрузчика
/dev	–	(devices) файлы устройств
/etc	–	(etcetera) конфигурационные файлы
/home	–	домашние каталоги пользователей
	–	(library) необходимые библиотеки общего
/lib		пользования и модули ядра
	–	содержит точки монтирования для съёмных
/media		носителей
	–	(mount) точка монтирования для временно
/mnt		монтируемой файловой системы
	–	(optional applications) дополнительное
/opt		программное обеспечение
/proc	–	(process) информация для процессов
/root	–	домашний каталог суперпользователя
/run	–	процессы
	–	(system binaries) системные исполняемые
/sbin		файлы

	-	(server) данные сервисов, предоставляемых системой
/srv		
/sys	-	(system) системная информация
/tmp	-	(temp) временные файлы
/usr	-	(user applications) программы пользователя
/usr/bin	-	исполняемые файлы пользователя
/usr/lib	-	библиотеки пользователя
/usr/local	-	файлы пользователя
/var	-	(variable) переменные файлы
/var/lib	-	базы данных
/var/lock	-	файлы блокировок
/var/log	-	файлы логов
/var/mail	-	почта
/var/run	-	PID процессов
/var/spool	-	принтер

С точки зрения безопасности должное внимание необходимо уделять следующим разделам: /etc, /bin, /sbin, /lib, которые содержат не только конфигурационные файлы, но и системные программы и библиотеки.

Основными конфигурационными файлами Linux являются:

/etc/profile Системные настройки окружения - скрипт, выполняющийся при старте системы.

/etc/crontab Список команд и времени их выполнения для запуска (назначенные задания для демона cron).

/etc/passwd База данных пользователей с полями, определяющий логин (username), реальное имя, домашний каталог (home directory), зашифрованный пароль (encrypted password, а точнее - хеш от него, вычисленный по известному алгоритму, например MD5) и другую информацию по каждому пользователю.

/etc/services Перечисляет сетевые сервисы, которые система поддерживает.

/etc/sudoers Список пользователей со специальными привилегиями для команд, которые они могут выполнять.

Технологии проверки целостности информации

Целостность информации (также целостность данных) – состояние информации (данных), при котором отсутствует любое ее изменение либо изменение осуществляется только преднамеренно субъектами, имеющими на него право [из п. 17 Приложения А ГОСТ Р 50922-2006].

Обнаружение ошибок – действие, направленное на контроль целостности данных при записи/воспроизведении информации или при её передаче по линиям связи. Исправление ошибок (коррекция ошибок) – процедура восстановления информации после чтения её из устройства хранения или канала связи.

Хеширование (иногда хэширование, англ. hashing) – преобразование входного массива данных произвольной длины в выходную битовую строку фиксированной длины. Такие преобразования также называются хеш-функциями или функциями свёртки, а их результаты называют хешем, хеш-кодом или дайджестом сообщения (англ. message digest).

Существующие алгоритмы:

Контрольные суммы:

CRC - (команда в Linux - cksum)

ХЭШ:

MD - MD5, MD6 (команды в Linux - md5sum, openssl)

SHA - SHA-1, SHA-2 (команды в Linux - sha1sum, openssl)

ГОСТ Р 34.11-94 - (rhash, openssl)

ГОСТ Р 34.11-2012 - (openssl)

Tiger - (rhash)

RIPEMD-160 - (rhash)

HAS-160 - (rhash)

EDON-R - (rhash)

Whirlpool - (rhash)

Snefru - (rhash)

Magnet-ссылки:

DC++ - (rhash)

BitTorrent - (rhash)

ED2K - (rhash)

AICH - (rhash)

Технологии резервирования и синхронизации информации

Для повышения надежности информационных систем часто используют распределенные системы, которые используют технологии резервирования и синхронизации информации.

Пример программ резервирования информации:

Bacula – кроссплатформенное клиент-серверное программное обеспечение, позволяющее управлять резервным копированием, восстановлением, и проверкой данных по сети для компьютеров и операционных систем различных типов.

Система построена по технологии клиент-сервер, и для передачи данных использует протокол TCP. Резервные копии создаются в собственном, полностью открытом формате.

Система резервирования данных Bacula состоит из четырёх основных элементов: Director Daemon, Storage Daemon, File Daemon и Bacula Console. Все эти элементы реализованы в виде самостоятельных приложений.

Director Daemon (DD) – это центральный элемент системы, осуществляющий управление её остальными компонентами. В его задачи входит управление процессом резервирования/восстановления данных, обеспечение интерфейса управления для администраторов и многое другое.

Говоря проще – это диспетчер, который инициирует все процессы и отслеживает ход их выполнения.

Storage Daemon (SD) – приложение, отвечающее за чтение/запись данных непосредственно на устройства хранения информации. Принимает управляющие команды от DD, а также резервируемые данные от/к File Daemon.

File Daemon (FD) – этот элемент ещё можно назвать Агентом. Ведь именно он работает в рамках операционной системы, данные которой необходимо резервировать. File Daemon выполняет всю рутину, осуществляя обращение к резервируемым файлам и их дальнейшую передачу к SD. Также на стороне FD выполняется шифрование резервных копий, если это определено конфигурацией.

Bacula Console (BC) – интерфейс администратора системы. По своей сути, это командный интерпретатор для управления Bacula. Строго говоря, Bacula Console может быть расширена с помощью графических систем управления, которые, как правило, являются всего лишь надстройкой над BC. К таким системам можно отнести Tray Monitor и Bat. Первая устанавливается на компьютере администратора системы и осуществляет наблюдение за работой системы резервирования, а вторая обеспечивает возможность управления посредством графического интерфейса.

Bacula Catalog – база данных, в которой хранятся сведения обо всех зарезервированных файлах и их местонахождении в резервных копиях. Каталог необходим для обеспечения эффективной адресации к требуемым файлам. Поддерживаются MySql, PostgreSQL и SQLite.

Такое структурное деление позволяет организовать очень гибкую систему резервирования, когда Storage Daemon разворачивается на выделенном сервере с несколькими устройствами хранения данных. Также Bacula Director может управлять несколькими экземплярами SD, обеспечивая резервирование части данных на одно устройство хранения, а части – на другое.

Amanda (Advanced Maryland Automatic Network Disk Archiver) является решением бэкапа, которое позволяет ИТ-администратору настраивать один главный сервер резервного копирования для восстановления нескольких хостов по сети на ленточные накопители, диски или оптические носители.

Amanda использует собственные утилиты и форматы (например, `dump` и `tar`) и может создавать резервные копии большого количества серверов и рабочих станций с несколькими версиями Linux или Unix.

dump - традиционная утилита резервного копирования для Unix-подобных операционных систем. Она может работать с десятью уровнями инкрементальных дампов, записывать их на стриммерные ленты, на любые блочные устройства и просто в файлы на другом разделе. Восстанавливать можно как раздел целиком, так и отдельные файлы, и директории. Позволяет архивировать данные на лету, используя различные алгоритмы сжатия. Какие именно - лучше справиться в мануале (`man dump`) на свою ОС. Для Linux это: `zlib`, `bzlib` и `lzo`. `Zlib` и `bzlib` поддерживают 9 уровней сжатия и, соответственно, разную скорость сброса дампа.

Программа синхронизации информации:

Rsync (англ. Remote Synchronization) – программа для UNIX-подобных систем, которая выполняет синхронизацию файлов и каталогов в двух местах с минимизацией трафика. Для синхронизации двух каталогов используют опцию `-a`:

```
rsync -a foobar_src/ foobar_dst/
```

В данном случае файлы и директории из источника скопируются в приемник, при этом, если там были файлы, то совпавшие по имени будут перезаписаны, а остальные не тронуты.

Задание к практической работе

Сброс пароля root в Linux

Если установлен загрузчик и к нему возможно получить доступ, тогда для получения доступа к загрузочному конфигурационному файлу необходимо:

1. Выбрать нужную операционную систему => нажать 'е' => найти строку, которая начинается с **linux** (linux 1.6, linuxefi);
2. В этой строке заменить *ro* на *rw* (право записи);
3. Дописать в конец строки после *quiet init=/bin/bash* (если диск зашифрован, то необходимо дописать *plymouth.enable = 0*);
4. Нажать F10 или Ctrl+x и в открывшейся консоли с помощью команды *passwd* можно изменить пароль входа в систему (если каталог доступен только для чтения, то можно его монтировать командой *mount -n -o remount, rw /*);
5. Перезагрузить машину */sbin/reboot -f*.

Проверка целостности (хеширования) информации

Для выполнения данного задания создаете пользователя или каталог с фамилией пользователя и все действия в данном режиме (под пользователем или внутри созданного каталога).

Приводите примеры утилит, реализующих функции целостности или хеширования информации:

CRC - (команда в Linux - *cksum*)

MD - MD5, MD6 (команды в Linux - *md5sum*, *openssl*)

SHA - SHA-1, SHA-2 (команды в Linux - *sha1sum*, *openssl*)

ГОСТ Р 34.11-94 - (*rhash*, *openssl*)

ГОСТ Р 34.11-2012 - (*openssl*)

Tiger - (*rhash*)

RIPEMD-160 - (*rhash*)

HAS-160 - (*rhash*)

EDON-R - (*rhash*)

Whirlpool - (*rhash*)

Snefru - (*rhash*)

Резервирование данных

Осуществите настройку резервирования (Bacula, Amanda, dump) и синхронизации (Rsync) информации с графическим сопровождения данного процесса.

Составьте отчет о выполнении практической работы.

Включите в него копии экрана пошагового выполнения задания и ответы на вопросы практической работы.